

A Sub-Milliwatt and Sub-Millisecond 3-D Gaze Estimator for Ultra Low-Power AR Applications

Sungmin Moon
Department of Electrical and
Computer Engineering,
Sungkyunkwan University
Suwon, Republic of Korea
sang6989@skku.edu

Chao Zhang
SAIT China Lab, Samsung Research
China - Beijing (SRC-B)
Beijing, China
c0502.zhang@samsung.com

Sooill Park
Department of Electrical and
Computer Engineering,
Sungkyunkwan University
Suwon, Republic of Korea
kor1625@skku.edu

Hui Zhang
SAIT China Lab, Samsung Research
China - Beijing (SRC-B)
Beijing, China
hui123.zhang@samsung.com

Woo-Shik Kim
Samsung Advanced Institute of
Technology (SAIT)
Suwon, Republic of Korea
wooshik.kim@samsung.com

Jong Hwan Ko
College of Information and
Communication Engineering,
Sungkyunkwan University
Suwon, Republic of Korea
jhko@skku.edu

ABSTRACT

The critical factors of real-time gaze tracking are high accuracy and user-friendliness such as low latency and no run-time calibration. Existing gaze estimator hardware designs are based on 2D regression algorithms as 2D methods have a simple computation process. However, they require multiple run-time calibration steps, and are vulnerable to head motions. On the other hand, the 3D model-based method can maintain better accuracy than the 2D method without run-time calibration steps, and is robust to head motions. In this paper, we aim to design the first 3D model-based gaze estimator hardware that consumes less than 1mW power and 1ms latency per frame. The simulation results based on the hardware synthesis show that the proposed design requires 172 μ W and 0.5ms per frame, while maintaining less than 0.9° error.

CCS CONCEPTS

• **Human-centered computing** → Ubiquitous and mobile computing; • **Hardware** → Integrated circuits.

KEYWORDS

Gaze Tracker; Gaze Vector Estimation; Smart Glasses; ASIC

ACM Reference Format:

Sungmin Moon, Chao Zhang, Sooill Park, Hui Zhang, Woo-Shik Kim, and Jong Hwan Ko. 2021. A Sub-Milliwatt and Sub-Millisecond 3-D Gaze Estimator for Ultra Low-Power AR Applications. In *Adjunct Proceedings of the 2021 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2021 ACM International Symposium on Wearable Computers (UbiComp-ISWC '21 Adjunct)*, September 21–26, 2021, Virtual, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3460418.3479360>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

UbiComp-ISWC '21 Adjunct, September 21–26, 2021, Virtual, USA

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8461-2/21/09...\$15.00

<https://doi.org/10.1145/3460418.3479360>

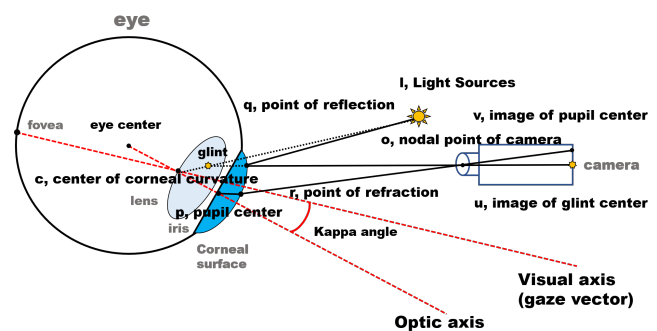


Figure 1: 3D model based gaze vector estimation method

1 INTRODUCTION

Rapid advancements in computing speed, digital video processing, and low-cost hardware brought gaze tracking equipment closer to consumers, with applications in gaming and virtual reality [5]. For head-mounted displays and smart glasses, there have been various algorithms that realize gaze vector estimation. The conventional video-based gaze tracking method, which is called PCCR (Pupil Center Corneal Reflection), uses at least one camera, near infra-red LEDs, and the monitor screen. This methodology calculates gaze vectors using the glints on the eye cornea surface produced by LEDs and pupil centers. Gaze estimation algorithms can be classified into two categories: 2D regression methods[5] and 3D model-based methods[2, 3]. There are pros and cons to each method in several aspects. Generally, 2D algorithms have simpler operations, but they are vulnerable to head motions. Another critical drawback of the 2D algorithm is that it requires a run-time *calibration*. Calibrations in 2D algorithms are executed whenever the gaze tracking session starts at the application stage to choose the gaze coefficient. The calibration process requires users to gaze at several targets on the monitor screen, while the camera captures the eye position. As the calibration should be performed before the user operates the gaze estimator, it greatly reduces the user-friendliness of the tracking

system. On the other hand, 3D algorithms require no run-time calibration, it may require the one-time offline calibration to obtain camera intrinsic/extrinsic parameters and the angle between the optical axis and the gaze vector, also known as the visual axis. In addition, 3D algorithms are more accurate than 2D algorithms, achieving around 1° angular error, while the latter achieving around 1-2°.

One of the weaknesses of 3D algorithms is that they tend to be more complex than 2D algorithms. It would be challenging to run a 3D algorithm software on an embedded system which would hardly satisfy the power consumption and latency requirement (in the order of milliseconds) in conventional AR/VR applications. For example, it is reported that the gaze tracker implemented in Raspberry Pi has a high power consumption of 2.4W due to accurate computer vision algorithms[12]. To support the complex gaze vector schemes in ultra-low power smart glasses, it should be implemented as hardware such as ASIC (Application-Specific Integrated Circuit). There have been a few studies on gaze estimator hardware design, but most of them are based on 2D algorithms which are easy to implement [4, 8, 11]. To the best of our knowledge, there have been no research work that implemented 3D algorithms on digital hardware.

In this paper, we propose the hardware that accelerates the complex computation of the gaze vector based on the 3D model algorithm. We aim to design ultra low-power and low-latency hardware that requires less than 1mW power and 1ms latency per one frame processing, which are critical requirements for resource-constrained smart glasses. At the same time, we target for high area-efficiency and accuracy. The simulation results based on hardware synthesis show that the proposed design requires 172μW power and 0.5ms latency with the area of 0.391mm², while maintaining the accuracy of the reference algorithm (less than 0.9° error).

2 BACKGROUND AND RELATED WORK

2.1 Gaze Estimation Algorithms

The gaze vector in 3D-based methods can be estimated using the mathematically modeled human eye shown in Figure 1. Guestrin et al. [3] classified the case of the configurations according to the number of cameras and LEDs. There are some existing 3D-based methods [1, 7], they use two cameras with LED arrays, 4 cameras with 2 LEDs using rotating mirrors, respectively. We choose Guestrin's work [3] which uses one camera settings capable of applying to AR devices. Above all, one camera and multiple light source configurations allow free head motion, and theoretically, a single calibration process is enough to obtain eye parameters. These parameters are person-specific, but common settings can be applied. Using the configurations of one camera and two light sources, we can let the coordinates of the nodal point of camera as $\mathbf{o}$, and the light sources as $\mathbf{l}_i$ ($i = 1, 2$). The 2D coordinates of the pupil center and two glints are obtained in the preprocessing stage. Using camera intrinsic and extrinsic parameters, they can be mapped to 3D vectors $\mathbf{v}$, $\mathbf{u}_1$, $\mathbf{u}_2$, respectively.

After preparing the above input coordinates, the whole procedure of the 3D method consists of 5 steps:

- (1) computation of the center of corneal curvature $\mathbf{c}$,
- (2) computation of the refraction point $\mathbf{r}$,

- (3) computation of the center of pupil $\mathbf{p}$,
- (4) computation of optical axis ω ,
- (5) and the computation of the visual axis $\mathbf{g}$.

Calculation of $\mathbf{c}$ uses the following equation:

$$\mathbf{c} = \mathbf{o} + k_c \mathbf{b} \text{ (for some } k_c). \quad (1)$$

The vector $\mathbf{c}$ and normal vector $\mathbf{b}$, which is the direction from $\mathbf{o}$ to $\mathbf{c}$, can be calculated using the fact that $\mathbf{c}$ is in the line of an intersection of the plane defined by $\mathbf{o}$, $\mathbf{l}_1$, $\mathbf{u}_1$, and $\mathbf{o}$, $\mathbf{l}_2$, $\mathbf{u}_2$. k_c means the distance from $\mathbf{c}$ to $\mathbf{o}$. To obtain k_c , it is required to use the relation of equations of scaling factor (2), and the point of reflection (3), and solve the equation (4) represented by the function of k_c , where R is a radius of corneal curvature.

$$k_{q,i} = k_c \frac{\mathbf{o} - \mathbf{u}}{\|\mathbf{o} - \mathbf{u}\|} \cdot \mathbf{b} - \sqrt{k_c^2 \left(\frac{\mathbf{o} - \mathbf{u}}{\|\mathbf{o} - \mathbf{u}\|} \cdot \mathbf{b} \right)^2 - k_c^2 + R^2} \quad (2)$$

$$\mathbf{q}_i = \mathbf{o} + k_{q,i} \frac{\mathbf{o} - \mathbf{u}_i}{\|\mathbf{o} - \mathbf{u}_i\|} \text{ for some } k_{q,i} \quad (3)$$

$$\left(\frac{\mathbf{l}_i - \mathbf{q}_i}{\|\mathbf{l}_i - \mathbf{q}_i\|} - \frac{\mathbf{o} - \mathbf{q}_i}{\|\mathbf{o} - \mathbf{q}_i\|} \right) \cdot (\mathbf{q}_i - \mathbf{c}) \quad (4)$$

In the second step, the point of refraction $\mathbf{r}$ is calculated by solving the equation (5). k_r can be acquired using $\mathbf{o}$, $\mathbf{c}$, and $\mathbf{v}$.

$$\mathbf{r} = \mathbf{o} + k_r (\mathbf{o} - \mathbf{v}) \text{ for some } k_r \quad (5)$$

$$\mathbf{p} = \mathbf{r} + k_p \mathbf{t} \text{ for some } k_p \quad (6)$$

The computation of reflection point $\mathbf{p}$ in equation (6) can be obtained by constant k_p and unit vectors $\mathbf{t}$, using ζ and η calculated using $\mathbf{v}$, $\mathbf{o}$ and $\mathbf{r}$, $\mathbf{c}$, respectively.

The optical axis of the eye can be obtained by solving the equation (7).

And finally, the gaze vector $\mathbf{g}$ can be obtained by solving (8) after calculating the angles θ and ϕ by using the equation (7).

$$\omega = \frac{\mathbf{p} - \mathbf{c}}{\|\mathbf{p} - \mathbf{c}\|} = [\cos \phi \sin \theta, \sin \phi, -\cos \phi \cos \theta] \quad (7)$$

$$\mathbf{g} = [\cos(\theta + \beta) \sin(\theta + \alpha), \sin(\phi + \beta), -\cos(\phi + \beta) \sin(\theta + \alpha)] \quad (8)$$

2.2 Gaze Estimator Hardware Design

A few recent studies presented hardware design of a gaze estimator, mostly as a system including not only the gaze estimator but also the front-end modules such as a pupil edge detector and glint detector.

Hong et al.[4] is a low-power object recognition system integrated with a single-chip gaze estimation sensor, which focused on real-time detection and power efficiency. The gaze estimation algorithm is 2D regression-based, and the gaze error during calibration is 0.4° with 9 calibration points. The gaze estimation process is calculated in the fixed-point logarithmic SIMD processor integrated on the chip with the throughput of 0.3GOPS, and 10mW of power consumption.

Yang et al.[11] has also implemented an ASIC solution based on the 2D estimation method. They implemented the MAC unit to map the pupil center image to the gaze point on the VR display. Under the configuration of the Xilinx Kintex-7 platform running

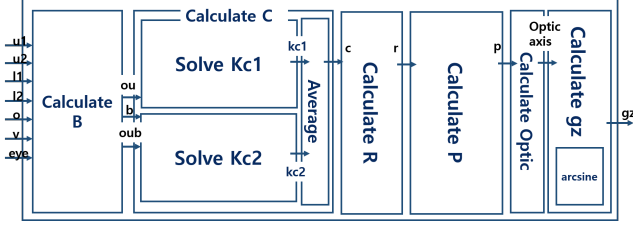


Figure 2: Block diagram of the proposed 3D gaze vector estimation system

on 200MHz frequency, they achieved the frame rate of 380Hz, and the error was under 1.5° .

Qin et al.[8] implemented a gaze tracker on the FPGA environment. The gaze tracker also calculates the gaze point by the mapping functions using the relationship of pupil center and glints based on the 2D method. This design was ported on a DE2-60 board with Altera FPGA running on 50MHz frequency and a NIOS-II processor. The detection rate is 11fps, and the error rate between FPGA results and simulation on PC was around 5%.

In addition, the Tobii integrated the EyeChipTM ASIC on the HTC Vive series [10]. The accuracy was around 0.5, and the sampling rate was 120Hz. However, they did not disclose detailed specifications that are required to perform the comparison. Moreover, since it is integrated into HMD VR devices, the hardware setup could be different.

3 DIGITAL 3D GAZE VECTOR ESTIMATOR

We propose a first full-digital design of a 3D gaze estimator that targets ultra low-power, low-latency, and area-efficient design. More specifically, we target power consumption under 1mW, latency under 1ms, and the area under 1mm^2 , which would be suitable for AR/VR applications with stringent power/latency/area constraints. While optimizing the design for high efficiency, we simultaneously target high accuracy with an error within 0.9° .

Figure 2 shows a block diagram of the proposed design, which generally follows the 3D gaze estimation algorithm described in the previous section. Based on this baseline algorithm, we first optimize the algorithm to make it suitable for hardware implementation. After that, we apply several hardware optimization techniques to further optimize the efficiency of the system.

3.1 Architecture Details

To follow the procedure of the 3D model-based method, the modules are designed by functional level. Compared to previous work, the percentage of data reuse of our method is low, integrating embedded SRAM into our design is unnecessary.

Our design focused on low power and area without accuracy loss, so the floating-point operations are chosen to maintain reference accuracy. However, those computations are complex for implementing to hardware, Synopsys DesignWare Building Block (DWBB) IP (Intellectual Property) are adopted for floating-point calculation.

For hardware input variables, it requires vector coordinates of two glints, pupil, two light sources, and the camera coordinates.

These inputs are all floating-point variables. In addition, the binary data is inputted containing about which eye. Describing details for architecture, each module corresponds to each equation described in the previous section. These modules consist of control signals and floating-point calculation blocks. Above all, the "Calculate C" module is the most complex part of the whole system. "SolveKc" module is implemented to solve complex "SolveKc" functions for each glint, it processes iterative loops to solve k_c for each glint. For the output, two k_c s are averaged and the final c will be calculated with MAC (Multiply-Accumulate) module and output to the next module. The final module "Calculate Gz" computes ϕ and θ using implemented arcsine module to estimate gaze vector.

3.2 Algorithm Optimization

In order to optimize the baseline algorithm while minimizing the accuracy loss, we apply following techniques.

3.2.1 SolveKc function. Then, we further optimize the calculation of k_c in equation (1), which is the most complex portion of the whole gaze vector estimation process. In equation (1), k_c is the only unknown variable, so it must be computed by substituting the equation (4) to the function of k_c by using the equation (2) and (3). This process results in a very complex function that is hard to solve with a hardware implementation. Therefore, to solve this monotonically increasing function, we can apply simple procedure. We can let the output of equation (4) as an error, then input the arbitrary number to k_c and repeat the process by increasing k_c with an arbitrary step size until the error approaches 0. When the sign of error changes, then reduce the step size and subtract from k_c to move backward. This process repeats until it converges to 0.

Furthermore, 10k images were evaluated to catch exceptional cases. It was found that the k_c had a close relationship with the value of unit vector $\mathbf{b}$. If a component of $\mathbf{b}$ had a large value, the k_c also became large. However, if the component of $\mathbf{b}$ becomes too large, there would be no solution or poor result for the whole gaze estimation. Therefore, we made an algorithm to predict a range of k_c with the value of $\mathbf{b}$. The initial value of k_c started from R, but the total loop iterations can be decreased by dynamically setting the start point of k_c using $\mathbf{b}$. In addition, an "early exit" was implemented to catch the error case if the value of k_c becomes too large. As a result, we save loop cycles about 30.3%.

3.2.2 Phi and Theta. Finally, to acquire ϕ and θ using arcsine from x, y value of optic axis, we initially had to calculate θ all over again if the equation: $-\cos\phi\cos\theta$ mismatch the z value of optic axis due to sign of phi. However, evaluation using 10k images, the approximation error was minimized by determining the sign of ϕ by simply subtracting ϕ from π .

3.2.3 Taylor approximation. DWBB IPs are capable of processing trigonometric functions such as sine or cosine, but inverse functions are not implemented to corresponding IPs. Therefore, we implemented the inverse trigonometric functions using the Taylor series approximation for more efficient design. Using software simulation, we first checked the accuracy degradation of Taylor approximation and determined the number of the terms. With several trials, $n = 4$ was enough for maintaining total accuracy with a small loss.

3.3 HW Optimization

The Synopsys Design Ware Building Block (DWBB) IP was used to easily implement the floating-point operations for accurate computation. DWBB IPs consists of basic add, multiply, division, square root, dot, MAC and even trigonometric functions. After algorithm optimization, the precision change to DWBB IPs was applied, which are flexible to precision changes. By synthesizing basic blocks, we checked dramatic reduction in both power and area about 70-75%.

However, due to complex floating-point operations, these IPs are heavy. Thus, for efficiency, reusing the basic block module can lead to power and area reduction. For minimization of the number of the basic blocks, we scheduled the timing of data flow. As a result, power and area have decreased due to the reduction of the total number of IPs, increasing hardware utilization. Besides, strict timing of data flow also led to saving latency.

4 EXPERIMENT & EVALUATION

4.1 Simulation Settings

For the parameter settings in the 3D method, we chose the camera intrinsic/extrinsic parameters first. we used the specifications of the Sony IMX219 CMOS image sensor. The sensor's diagonal size is 4.6mm, and the resolution of the sensor is 3296x2480, which means there are 4124pixels in the diagonal. Therefore, we can set the 1 pixel as 0.0011mm.

We used the general settings for the human eye parameter. R, the radius of corneal curvature is 7.8mm. K, the distance between the pupil center and the center of corneal curvature is 4.2mm. Refraction parameters n_1, n_2 are 1.3375 and 1, respectively. α and β , the horizontal and vertical component of kappa angle, is 5° (-5° for right eye) and 1.5° , respectively.

We used the 10k images of the NVGaze dataset [6] to generate pupil and glint coordinates using the PuRe algorithm [9] implemented in software, then using as inputs for the gaze vector estimator.

Table 1: Synthesis Result

Module	Area (μm^2)	Dynamic Power (μW)	Leakage Power (μW)
Calculate B	72629.4	32.197	0.299
Calculate C	197593.1	87.828	0.844
Calculate R	32814.7	11.351	0.143
Calculate P	47672.6	23.515	0.204
Calculate Optic	14023.8	9.046	0.060
Calculate Gz	25936.8	9.460	0.116
Total Module	391,334.7	170.164	1.671

4.2 RTL Simulation

To verify that the output from the hardware design is equivalent to the reference software simulation, we performed an RTL simulation using a Xilinx Vivado simulator. Consequently, using floating-point calculation with DWBB IPs, we were able to get the result 100% equal to the software result.

Table 2: Performance Comparison

	JSSC'16[4]	This work	
		Before opt.	After opt.
Chip function	Pupil + Gaze estimation	Gaze estimation	
Algorithm	2D regression	3D model	
CMOS process (nm)	65	28	
Frame rate (fps)	30	2000	
Area (mm^2)	$11.29^1 (1.134)^2$	1.499	0.391
Power	10mW	387.6 μW	171.8 μW
Error	0.5°	0.9°	

¹ Total chip ² RISC processor

4.3 HW Synthesis

We performed the hardware synthesis with Synopsys Design Compiler. We set 400ns (2.5MHz) as the clock period, and we used a 28nm technology library. Power and area breakdown results can be found in Table 1. Then we compared with 65nm CMOS gaze estimator processor[4], which is the only work with power and area results made available together. Also, we compared the synthesis result before and after applying the algorithm optimization to our work. The comparison details is shown in Table 2.

For the area, our design requires 0.391mm². It is 65.5% smaller than the previous work [4] that requires 1.134mm². In terms of power consumption, the design consumes 170.164 μW of dynamic power and 1.671 μW of leakage power. On the other hand, the total chip of previous work [4] had a power consumption of 10mW. Also, using the clock period of 400 μs , we can get the latency of 0.5ms, resulting in 2000 frames per second throughput.

5 CONCLUSION

We are proposing the first 3D-based gaze vector estimator hardware work based on the 3D method with sub-milliwatt power and sub-millisecond resulting in area-efficient enough to embed to smart glasses maintaining the accuracy of reference algorithms.

6 FUTURE WORK

Our next step will be synthesizing hardware implementation on the FPGA environment.

ACKNOWLEDGMENTS

This work was supported by Samsung Advanced Institute of Technology.

REFERENCES

- [1] David Beymer and Myron Flickner. 2003. Eye gaze tracking using an active stereo head. In *2003 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2003. Proceedings.*, Vol. 2. IEEE, IEEE Computer Society, 1730 Massachusetts Ave., NW Washington, DC United States, II-451.
- [2] Elias Daniel Guestrin. 2010. *Remote, non-contact gaze estimation with minimal subject cooperation*. University of Toronto, 395 Wellington Street, Ottawa ON K1A 0N4, Canada.
- [3] Elias Daniel Guestrin and Moshe Eizenman. 2006. General theory of remote gaze estimation using the pupil center and corneal reflections. *IEEE Transactions on biomedical engineering* 53, 6 (2006), 1124–1133.
- [4] Injoon Hong, Kyeongryeol Bong, Dongjoo Shin, Seongwook Park, Kyuho Jason Lee, Youchang Kim, and Hoi-Jun Yoo. 2015. A 2.71 nJ/pixel gaze-activated object recognition system for low-power mobile smart glasses. *IEEE Journal of Solid-State Circuits* 51, 1 (2015), 45–55.

- [5] Anuradha Kar and Peter Corcoran. 2017. A review and analysis of eye-gaze estimation systems, algorithms and performance evaluation methods in consumer platforms. *IEEE Access* 5 (2017), 16495–16159.
- [6] Joohwan Kim, Michael Stengel, Alexander Majercik, Shalini De Mello, David Dunn, Samuli Laine, Morgan McGuire, and David Luebke. 2019. Nvgaze: An anatomically-informed dataset for low-latency, near-eye gaze estimation. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, 2 Penn Plaza, Suite 701, New York, New York 10121-0701, USA, 1–12.
- [7] Takehiko Ohno and Naoki Mukawa. 2004. A free-head, simple calibration, gaze tracking system that enables gaze-based interaction. In *Proceedings of the 2004 symposium on Eye tracking research & applications*. ACM, 2 Penn Plaza, Suite 701, New York, New York 10121-0701, USA, 115–122.
- [8] Huabiao Qin, Xiangmin Xu, Zongwei Hu, and Dongyang Zhang. 2014. Eye tracking system based on SOPC. In *2014 21st IEEE International Conference on Electronics, Circuits and Systems (ICECS)*. IEEE, IEEE, 445 Hoes Lane, Piscataway, New Jersey, USA, 171–174.
- [9] Thiago Santini, Wolfgang Fuhl, and Enkelejda Kasneci. 2018. PuRe: Robust pupil detection for real-time pervasive eye tracking. *Computer Vision and Image Understanding* 170 (2018), 40–50.
- [10] TobiiAB. 2017. *Tobii Pro VR integration*. TobiiAB. <https://www.tobiiipro.com/product-listing/vr-integration/>
- [11] Quan Yang, Gaoming Sun, Yang Gao, Zhenkun Song, and Hongzhen Xue. 2018. 30-2: Implementation of a Real-time Eye Gaze Tracking Solution for ASIC Based on VR Display. In *SID Symposium Digest of Technical Papers*, Vol. 49. Wiley Online Library, Wiley, 111 River Street Hoboken, NJ 07030 USA, 385–387.
- [12] Lan Zhang, Xiang-Yang Li, Wenchao Huang, Kebin Liu, Shuwei Zong, Xuesi Jian, Puchun Feng, Taeho Jung, and Yunhao Liu. 2014. It starts with igaze: Visual attention driven networking with smart glasses. In *Proceedings of the 20th annual international conference on Mobile computing and networking*. ACM, 2 Penn Plaza, Suite 701, New York, New York 10121-0701, USA, 91–102.